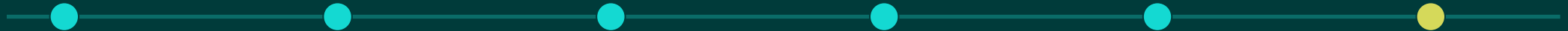

APPENDIX C

Version Control and Git

One-block overview: project history, commits, and the local Git workflow.

Edit → Review → Stage → Commit →
Continue



What students should leave with

Git awareness, not Git expertise.

Why version control exists
Stop relying on “final2” folders.

What Git records
Project snapshots with context.

The local workflow
Edit, review, stage, commit, repeat.

Remote vocabulary
Understand clone, pull, push, and
branches.

The beginner problem

Manual backups preserve files, but not useful history.

Copy-folder versioning

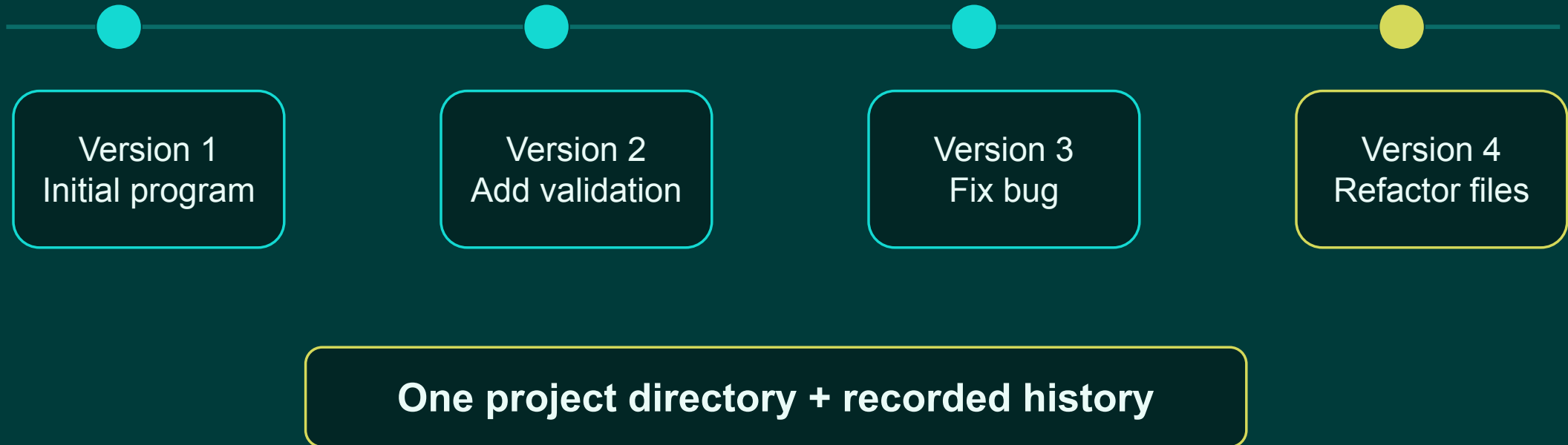
project/
project_old/
project_backup/
project_final/
project_final2/

Questions still remain

Which version worked?
What changed?
Why did it change?
Can I safely experiment?

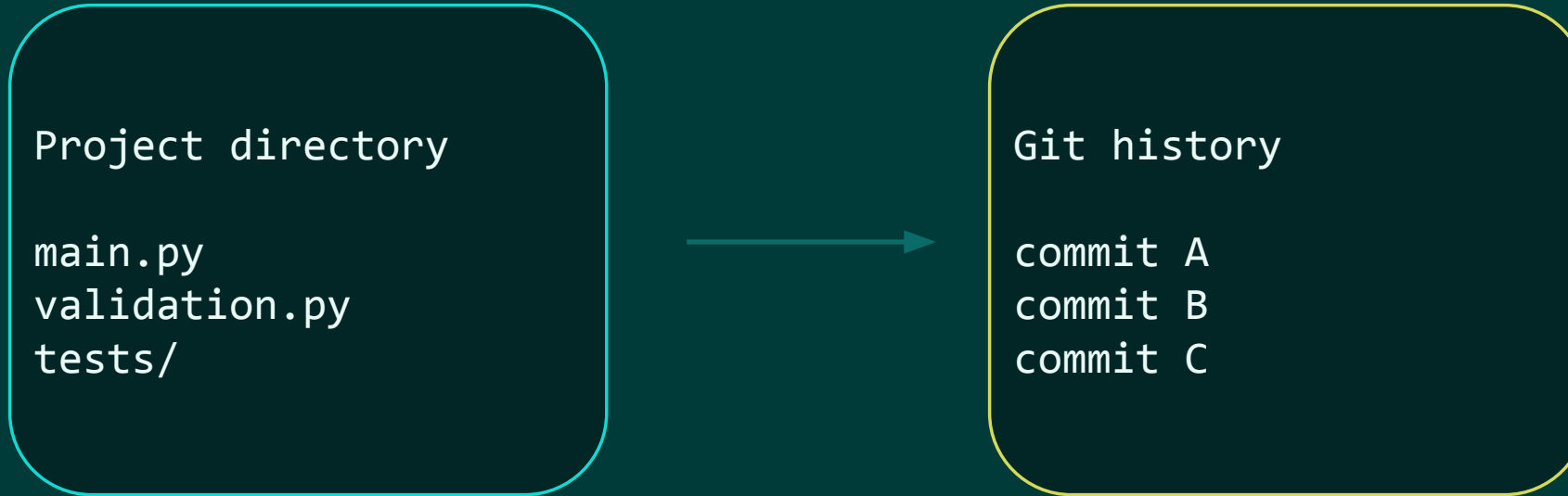
Version control is project history

A controlled sequence of meaningful checkpoints.



What is Git?

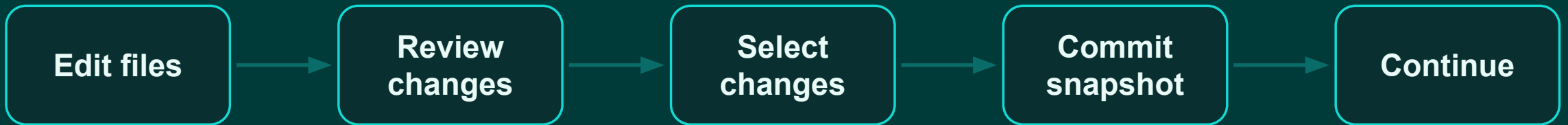
Git is local version control software.



Repository = project files plus Git history

Git does not save every keystroke

The developer decides when changes become history.



A commit is a named checkpoint, not an autosave.

The three-zone mental model

Working directory → staging area → repository history.



Starting and inspecting a repo

``git init`` creates a repository; ``git status`` asks what Git sees.

```
record_processor/  
├── main.py  
├── validation.py  
└── tests/
```

```
$ git init  
Initialized empty Git repository  
  
$ git status  
Untracked files:  
  main.py  
  validation.py  
  tests/
```

``git status`` is the “what is happening?” command.

Reviewing changes with git diff

Before recording history, inspect what changed.

```
$ git diff
- return score > 0
+ return 0 <= score <= 100

# Removed line begins with -
# Added line begins with +
```

Use diff to compare

Previous version

↓
Current files

Goal: understand the change before committing it.

Staging with git add

`git add` selects changes for the next commit.



```
$ git add validation.py
```

```
$ git status
```

```
Changes to be committed:
```

```
  modified: validation.py
```

Think: “include this file’s current changes in the next snapshot.”

Committing a snapshot

A commit creates a named checkpoint in the repository history.

```
$ git commit -m "Add score validation"
```

```
[main 3f4a21c] Add score validation  
1 file changed, 2 insertions(+), 1 deletion(-)
```

Commit message

Add score validation

Explains why this checkpoint exists.

Commit = selected changes + message + point in history

Commit messages should explain intent

Future you is also a reader of the project history.

Weak messages

stuff
changes
update
fixed things
asdf

Better messages

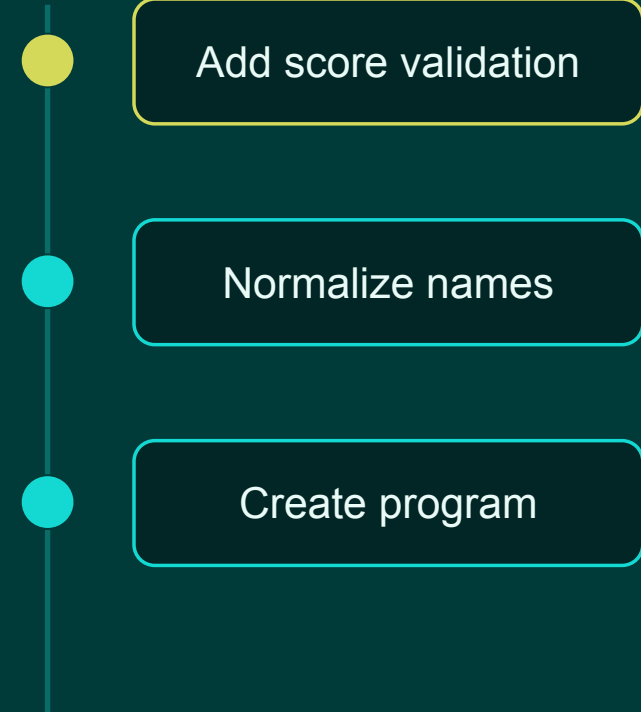
Add score validation
Normalize status values
Handle missing record IDs
Fix completion calculation

Prefer a concise sentence about the meaningful change.

Reading history with git log

`git log` shows the sequence of commits.

```
$ git log --oneline  
3f4a21c Add score validation  
87d19bb Normalize names before greeting  
4c82f01 Create greeting program
```



The complete local workflow

Most beginner Git work follows this loop.



```
$ git status  
$ git diff  
$ git add validation.py  
$ git commit -m "Add record validation"
```

When unsure, start with git status.

Ignoring generated files

Use `.gitignore` for files that do not belong in history.

`.gitignore`

`.venv/
__pycache__/
.pytest_cache/`

Commit this

`.gitignore`

It describes the
project.

Do not commit this

`.venv/
__pycache__/`

Generated locally.

Never commit passwords, API keys, or private credentials.

Git is not GitHub

Git can work completely on one computer.

Git

Version control software

Tracks project history
Creates commits
Runs locally

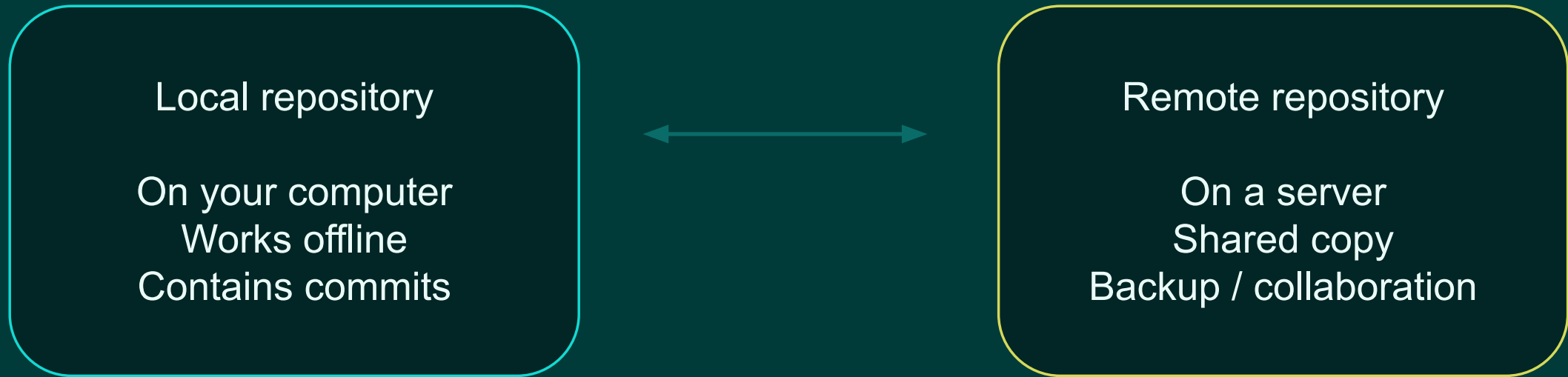
GitHub / GitLab

Online repository hosting

Stores remote copies
Supports sharing
Adds collaboration tools

Local and remote repositories

A project can have history on your computer and on a server.



Network synchronization connects them; Git history exists in both places.

Clone, pull, and push

Three remote vocabulary words students should recognize.

clone

Remote → Local
Create a local copy

pull

Remote → Local
Bring down changes

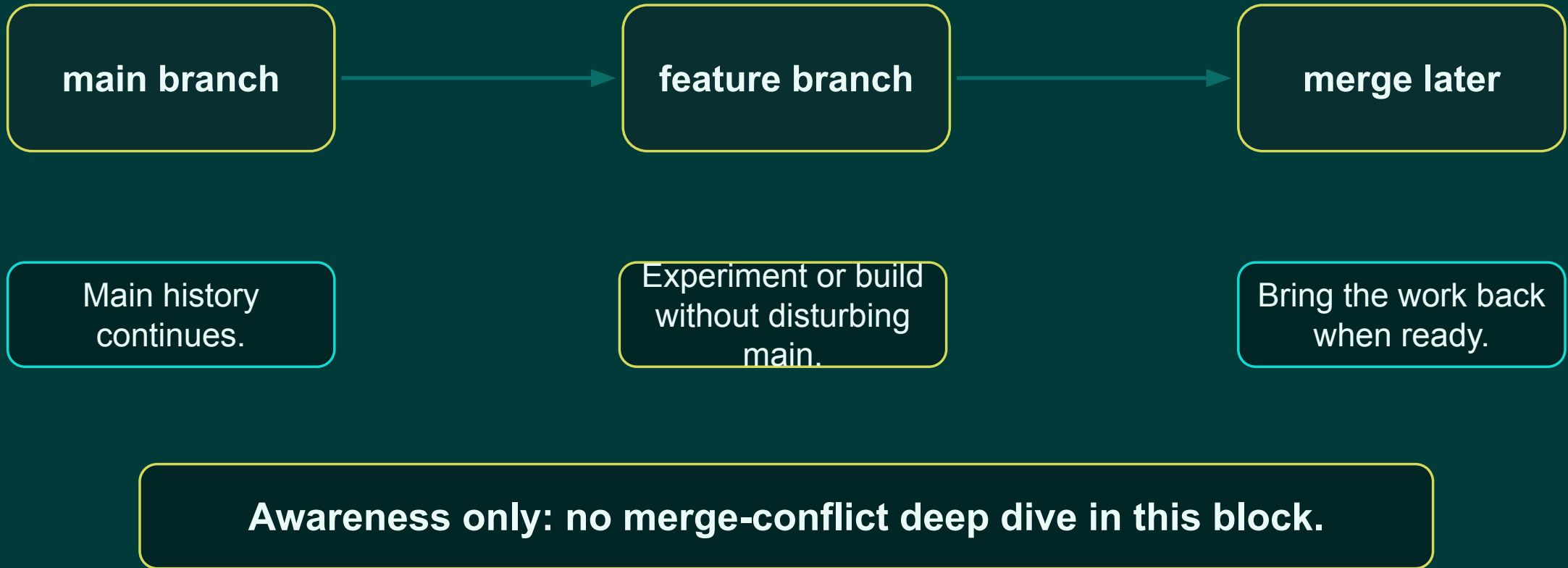
push

Local → Remote
Send your commits

Simple collaboration loop: pull latest → edit → commit → push

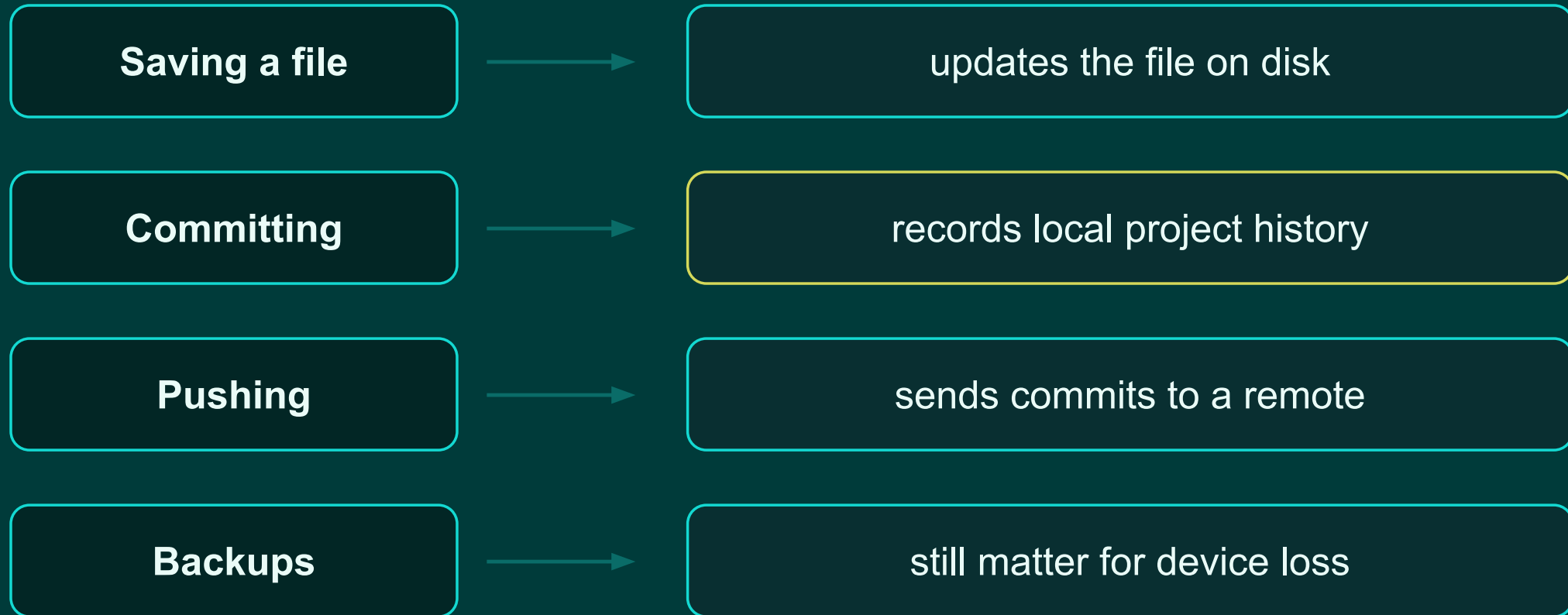
Branches: awareness only

Branches allow a separate line of work.



Common misunderstandings

These distinctions prevent many beginner Git frustrations.



Guided exercise setup

Create a tiny project and let Git record its story.

```
git_practice/  
└── main.py
```

```
def greet(name):  
    return f"Hello, {name}"  
  
print(greet("Alice"))
```

30 minutes: work the loop, not every Git option.

Exercise 1: initialize and commit

Create repository history for the first time.

```
$ git init  
$ git status  
$ git add main.py  
$ git commit -m "Create greeting program"  
$ git status
```

Student task

Identify untracked files.
Explain what changed after add.
Explain what changed after
commit.

Exercise time: 7 minutes

Exercise 2: modify and inspect

Change the code, then ask Git what changed.

```
def greet(name):  
    cleaned_name = name.strip()  
  
    return f"Hello, {cleaned_name}"
```

```
$ git status  
$ git diff  
$ git add main.py  
$ git commit -m "Normalize names  
before greeting"
```

Student task

What file changed?
What lines changed?
Was it committed before add?
Was it committed after commit?

Exercise time: 12 minutes

Exercise 3: add .gitignore

Commit project instructions, not generated files.

```
.gitignore  
  
.venv/  
__pycache__/  
.pytest_cache/
```

```
$ git status  
$ git add .gitignore  
$ git commit -m "Ignore generated Python  
files"  
$ git log --oneline
```

Discussion prompt

Why commit .gitignore but not .venv/?

Applied challenge

Use Git while making one small Python improvement.

1

Choose a tiny change

Example: add a validation
function

2

Review it

Use git status and git diff

3

Commit it

Use a meaningful message

```
$ git status  
$ git diff  
$ git add <file>  
$ git commit -m "Describe the change"
```

Goal: build the habit, not memorize every command.

End-of-block outcome

Students now have the mental model needed to begin using Git.



Knowledge check

What does git status tell you?
What does git add do?
What is a commit?
How is Git different from GitHub?
What should never be committed?

Git keeps the story of how the project reached its current state.